A COURSE-BASED PROJECT

On

Online Banking Transaction Pattern Analysis

Submitted in partial fulfillment of Big Data Analytics Lab

GRIET Lab On Board(G-LOB)

By

Name: M. Deepti **Roll No:23241A6746**

Name: N. Sanjana **Roll No:23241A6747**

Name: B. Bhavitha **Roll No:24245A6703**

Name: Safura Anjum **Roll No:24245A6706**

Department of Data science

GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Bachupally, Kukatpally, Hyderabad, Telangana, India,500090

2025-2026

GOKARAJU RANGARAJU
INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)
Hyderabad-500090

CERTIFICATE

This is to certify that the GLOB titled "**Online Banking Transaction Pattern Analysis**" is submitted by **M. Deepti (23241A6746), N. Sanjana (23241A6747), B. Bhavitha (24245A6703) and Safura Anjum(24245A6706)** in partial fulfillment of the award of degree in BACHELOR OF TECHNOLOGY in Computer Science and Engineering (Data Science) during Academic year 2025-2026.

Internal Guide

Ms. S. Srilatha (Assistant Professor)

Dr. M. Kiran Kumar (Professor)

Abstract

In the era of digital banking, financial institutions are increasingly reliant on data analytics to gain insights into customer behavior and optimize services. This project, titled **"Online Banking Transaction Pattern Analysis Using Spark and Hive,"** aims to analyze banking transaction data to uncover trends in deposits and withdrawals over time. Leveraging the power of big data tools such as Apache Spark and Apache Hive, the system ingests and processes a large volume of transaction records to identify monthly financial activity patterns. The dataset, simulated to represent real-world online banking transactions, includes customer IDs, timestamps, transaction types (deposit or withdrawal), amounts, and locations. The data is loaded into a Hive table using SQL-based queries for ease of querying and schema management. Spark is then utilized to perform large-scale data processing and time-based aggregation of transaction trends. The analysis focuses on extracting actionable insights, such as fluctuations in customer transaction behavior across months, helping to inform financial forecasting and fraud detection strategies. This end-to-end pipeline is implemented in a Windows environment using Visual Studio Code and the Spark & Hive Tools extension, offering an efficient and interactive development workflow. The results, presented through tabular summaries and optional visualizations, demonstrate how scalable data frameworks can support meaningful financial analytics in modern banking systems.

Table of Contents

Chapter No	Chapter Name	Page No
1	Introduction	
	1.1 Introduction	1
	1.2 Significance of the project	1
2	Literature Survey	
	2.1 Existing Approaches	2
	2.2 Drawbacks of Existing Approaches	3
3	Problem Statement	
	3.1 Problem Statement	4
	3.2 Objectives of the Project	5
	3.3.1 Architecture Diagram	6
	3.3.2 Software Requirements	8
	3.3.3 Hardware Requirements	9
4	Implementation	10-11
5	Results and discussion	
	5.1 Dataset	12
	5.2 Explanation of Results	14
	5.3 Results	15
6	Conclusion	19
7	References	20

Introduction

1.1 Introduction

The rapid evolution of digital technologies has significantly transformed the banking industry, particularly in the domain of online financial services. As millions of transactions are processed daily through digital channels, vast amounts of data are generated, creating new opportunities—and challenges—for financial institutions seeking to optimize operations and enhance customer experience. Understanding transaction behavior, especially the patterns of deposits and withdrawals, is crucial for banks to manage liquidity, forecast demand, detect anomalies, and tailor personalized services. Traditional methods of analyzing financial data often struggle with scalability and real-time performance. To address these limitations, this project employs a modern big data architecture using Apache Spark and Apache Hive, two widely adopted tools in data engineering and analytics. Spark provides in-memory distributed data processing for fast computation, while Hive offers a familiar SQL-like interface to manage structured datasets over Hadoop-based storage systems.

This project aims to analyze online banking transaction trends—specifically deposits and withdrawals—over time. A simulated dataset is ingested and processed using Hive and Spark in a Windows development environment configured through Visual Studio Code. The project demonstrates how transaction data can be aggregated by month and transaction type to reveal insights into customer behavior and financial activity cycles. By building this analytical pipeline, the project highlights the effectiveness of big data tools in deriving business intelligence from high-volume transactional datasets.

1.2 Significance of the Project

- **Improves Financial Insights:** Helps banks identify seasonal or monthly trends in deposits and withdrawals, enabling better cash flow forecasting and resource allocation.
- **Supports Fraud Detection:** Unusual or unexpected transaction patterns can indicate fraudulent activity, which can be flagged using trend analysis.
- **Customer Behavior Analysis:** Enables deeper understanding of customer habits and financial needs, leading to more personalized banking services and targeted marketing.
- **Data-Driven Decision Making:** Provides actionable intelligence to financial managers and analysts using scalable, real-time big data tools.
- **Demonstrates Big Data Integration**: Showcases how Apache Spark and Hive can be used together for efficient, large-scale financial data processing and analysis.

Literature Survey

2.1 Existing Approaches

1. **Statistical Time Series Analysis:** Traditional methods such as ARIMA and exponential smoothing are used to model and forecast transaction volumes over time, but they are limited in scalability and adaptability to non-linear trends.

2. **Machine Learning for Transaction Classification:** Supervised learning algorithms (e.g., Decision Trees, SVM, Random Forest) are applied to classify transaction types, detect anomalies, or predict customer behavior based on historical data.

3. **Data Warehousing with SQL-Based Analytics:** Data is often stored in relational databases or data warehouses and analyzed using SQL queries, which works well for structured data but lacks performance at scale or in real-time processing.

4. **Hadoop-Based Batch Processing:** Using MapReduce and HDFS, some systems process large transaction logs in batch mode. This approach is reliable but slower compared to modern in-memory frameworks like Spark.

5. **Real-Time Stream Processing :** Technologies like Apache Kafka and Spark Streaming are employed for real-time monitoring of transactions, fraud detection, and alert generation, enabling immediate insights.

6. **Clustering Techniques for User Segmentation :** Algorithms like K-Means and DBSCAN are used to segment customers based on transaction frequency, volume, and type, which aids in targeted financial product recommendations.

7. **Association Rule Mining for Pattern Discovery :** Techniques like Apriori or FP-Growth are used to uncover co-occurrence relationships between different types of transactions or spending categories.

8. **Deep Learning for Anomaly Detection:** Neural networks (e.g., Autoencoders, LSTMs) are employed to model complex transaction behaviors and detect outliers that may indicate fraud or abnormal activity.

9. **Big Data Architectures with Hive & Spark :** A growing number of studies leverage Spark for in-memory processing and Hive for SQL-based data handling, allowing both scalability and usability in analyzing financial data.

2.2 Drawbacks of Existing Approaches

- **Steep Learning Curve :** Requires understanding of multiple tools (Spark, Hive, Hadoop, HDFS) and their configurations, especially challenging for beginners.

- **Resource-Intensive Setup :** Running Spark and Hive locally or on VMs consumes significant CPU and memory, making it less ideal for machines with limited specs.

- **Batch-Oriented Nature of Hive :** Hive is optimized for batch processing, making it less suitable for real-time transaction analysis or fraud detection.

- **Latency in Query Execution :** Queries in Hive over large datasets may suffer from high latency compared to more optimized OLAP engines.

- **Manual Data Engineering** : Requires manual steps for data ingestion, transformation, and schema management; this can introduce human error or inefficiencies.

- **Poor Windows Compatibility:** Many components (HDFS, Hadoop daemons) are natively built for Unix-based systems; running them on Windows often involves workarounds or WSL.

- **Limited Real-Time Capabilities:** The pipeline is not designed for streaming data; it does not handle real-time transaction ingestion or immediate trend reporting.

- **Data Redundancy and Storage Overhead:** Without proper optimization, intermediate and temporary files stored in HDFS can result in excessive storage usage.

- **Security and Access Control Gaps :** Default setups lack robust security policies, which are critical for banking data involving personally identifiable information (PII).

- **Scalability Requires Cluster Deployment:** Truly scalable performance needs deployment on a distributed cluster, which adds cost and infrastructure complexity.

Proposed Model

3.1 Problem Statement

The rapid rise of digital banking has significantly transformed the way financial institutions handle transactions, leading to the generation of vast amounts of data daily. Each transaction—whether a deposit, withdrawal, or transfer—provides valuable insights into customer behavior, financial health, and business operations. However, as transaction volumes continue to grow, so does the complexity of managing, processing, and analyzing this data effectively. With the increase in digital transactions, financial institutions are finding it challenging to extract meaningful patterns that can inform decision-making in real-time, support predictive models, and assist in fraud detection. Current banking systems struggle with the sheer volume and speed at which data is generated, leading to inefficiencies and missed opportunities for timely insights.

Traditional database management systems (DBMS) and data warehouses, such as relational databases, are often ill-equipped to handle the high velocity of transactional data in online banking. These systems typically work well with structured data but become bottlenecks when faced with unstructured or semi-structured data, which is becoming more prevalent in the digital banking ecosystem. While solutions like Apache Hive, based on Hadoop, offer the capability to process large datasets, they primarily focus on batch processing and are not optimized for real-time data ingestion or low-latency analytics. Apache Spark, on the other hand, provides a more robust platform for real-time, in-memory data processing but requires integration with various tools and significant infrastructure setup. This combination of technologies can offer scalability and flexibility, yet many organizations still face challenges with seamless integration, data ingestion pipelines, and achieving real-time analytics. Furthermore, the ability to perform complex analytics while ensuring the security and privacy of sensitive banking data is another layer of complexity. This highlights the need for a more integrated and efficient approach that supports both batch and real-time processing for comprehensive, actionable insights into banking transaction patterns, customer behavior, and fraud detection.

3.2 Objectives of the Project

Data Ingestion and Storage Optimization
To ingest large volumes of banking transaction data into a distributed Hadoop ecosystem, ensuring the data is properly stored in HDFS. This objective focuses on optimizing storage mechanisms, ensuring efficient data parsing, cleaning, and structuring for further analysis.

Data Preprocessing and Transformation
To implement preprocessing and transformation tasks using Hive and Spark to prepare raw transaction data (withdrawals and deposits) into structured formats. This will include cleaning the data, removing duplicates, handling missing values, and converting data types to align with analysis requirements.

Trend Analysis Using Spark.
To identify and analyze transaction patterns such as withdrawal and deposit trends over time using Apache Spark. This includes leveraging Spark's distributed computing capabilities to perform real-time and batch-based data analysis at scale, producing insights on transaction volume, peak periods, and customer behavior.

Query Optimization in Hive
To implement optimized SQL-based queries using Apache Hive for large-scale transaction data stored in HDFS. The focus will be on efficient query execution, minimizing query runtime, and ensuring that both batch and interactive queries are processed swiftly to enable timely decision-making.

Fraud Detection and Anomaly Analysis
To develop models using Spark for detecting anomalies or fraudulent transactions by analyzing withdrawal and deposit trends. This will involve building algorithms capable of identifying unusual patterns, flagging suspicious activities, and integrating these insights into real-time reporting dashboards.

Data Visualization and Reporting
To design and implement dashboards and visual reports that allow users to easily interpret and interact with banking transaction trends. Using tools like Apache Zeppelin or Tableau, the objective is to present insights, such as peak transaction times, average transaction values, and potential fraudulent behavior in an accessible format.

3.3.1 Architecture Diagram

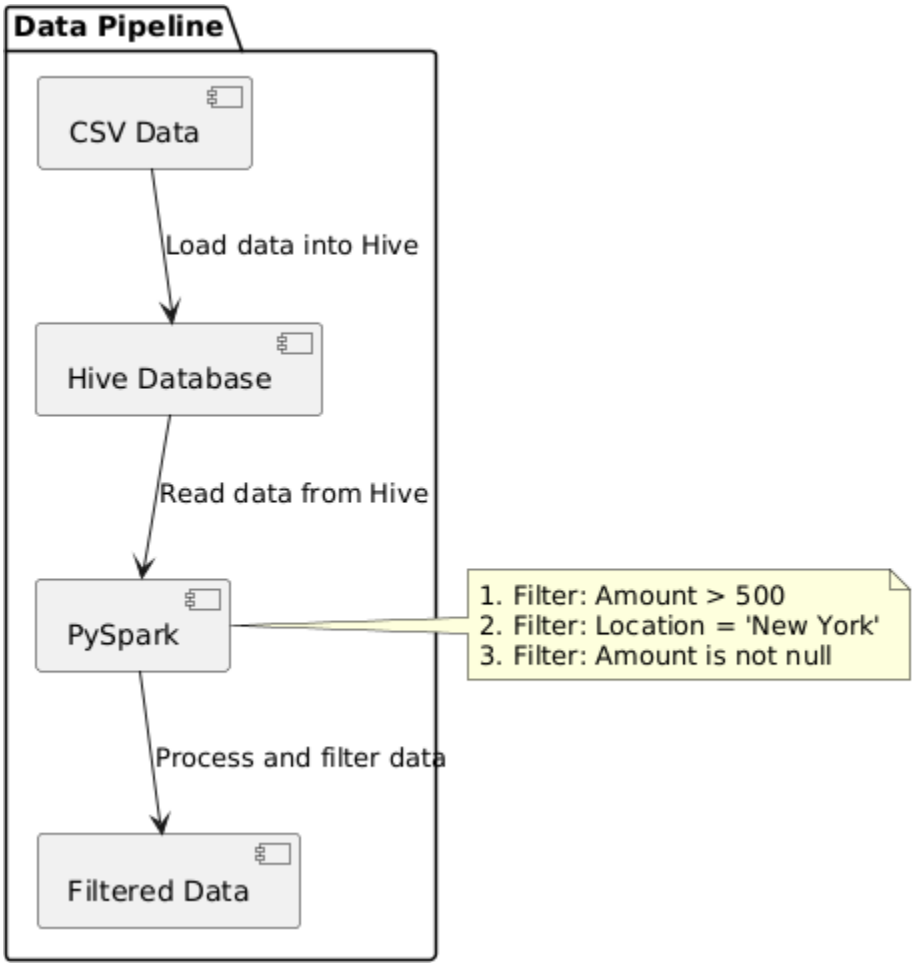

Fig. 1: Architecture Diagram

The architecture of a data pipeline using Hive and PySpark, as shown in the given example, focuses on the process of handling and analyzing large-scale banking transaction data. This system incorporates both Hive for data storage and management, as well as PySpark for data processing, filtering, and analysis. Here's a detailed breakdown of the architecture and its components:

Hive Database

The architecture begins with **Hive**, a data warehousing system built on top of Hadoop. Hive provides an SQL-like interface for querying large datasets stored in Hadoop Distributed File System (HDFS). In this system, Hive is responsible for storing and managing the banking transaction data. The first step in the pipeline involves creating a table in Hive to hold the transactions. The table is designed with several columns such as customer ID, transaction type, amount, location, and timestamp. This table allows structured querying of the data, facilitating efficient analysis.

Hive uses a **TextFile format** for storing the data, which is ideal for storing unstructured or semi-structured data. The data is loaded into the table using the **LOAD DATA** command, which points to a **CSV file** (data.csv) containing the raw transaction data. Once the data is loaded into Hive, it is organized in the table for easier querying and management.

CSV Data

The data that is loaded into Hive is originally in **CSV format**, which is a common format for storing tabular data. In the case of the banking transaction system, the CSV file contains records of transactions made by different customers. Each record contains fields such as customer ID, timestamp, transaction type (e.g., withdrawal, deposit), the transaction amount, and the location of the transaction. The raw CSV data serves as the input for the system, and it's imported into Hive for structured processing and querying.

PySpark

Once the data is loaded into Hive, the next part of the system involves **PySpark**, a Python API for Apache Spark. PySpark is used for large-scale data processing and analysis, allowing the system to handle vast amounts of banking transaction data efficiently.

The PySpark component begins by reading the CSV data into a **DataFrame**. This is done using the spark.read.csv() method, which loads the CSV data into PySpark and automatically infers the data schema (e.g., determining the data types of each column). The DataFrame structure allows for easy manipulation and analysis of the data, leveraging the distributed computing power of Apache Spark.

Filtering Data

The main operation in the PySpark component is **filtering** the data. The pipeline applies multiple filters to the banking transaction data to perform different analysis steps:

1. **Filtering Transactions Greater Than 500**: The first filter keeps only the transactions where the transaction amount is greater than 500. This filter can be useful for identifying high-value transactions or focusing on a specific range of transaction amounts.
2. **Filtering by Location**: After filtering for high-value transactions, the data is further refined by selecting only transactions that occurred in a specific location, such as "New York". This allows the system to focus on transactions that meet both criteria (high-value and specific location).
3. **Ensuring Non-Null Amounts**: Finally, a filter is applied to remove any transactions where the amount is **null**. This ensures that the analysis is performed on valid data, as transactions with null amounts would be irrelevant for the analysis.

Data Flow and Integration

The data pipeline starts by reading the CSV data and loading it into Hive for storage. PySpark then reads the data from Hive and applies a series of filters to extract valuable insights. This architecture is efficient because Hive is designed for handling large-scale data storage, while PySpark is optimized for distributed data processing. By using both Hive and PySpark, the system can efficiently manage and analyze large amounts of banking transaction data in a scalable and flexible manner.

In summary, the architecture combines Hive for data storage with PySpark for data processing, enabling large-scale analysis of banking transactions. By applying filters at various stages, the system ensures that only relevant data is processed and analyzed, which is crucial for business insights or decision-making. The use of Hive and PySpark in tandem makes the system highly scalable, efficient, and well-suited for big data analytics.

3.3.2 Software Requirements

Operating System:

- Windows, Linux, or macOS.

Programming Languages:

- Python (Version 3.8 or higher).

Machine Learning Framework:

- **Apache Spark:** For large-scale data processing and analysis.
- **scikit-learn:** For building predictive models and evaluating performance.

Development Environment:

- **Python IDE/Editor:** Any Python-compatible IDE (e.g., PyCharm, VS Code, Spyder).
- **Version Control:** Git (recommended for tracking changes and collaboration).

External Tools:

- **Hadoop Ecosystem:** Including HDFS for distributed storage, and Hive for querying data.
- **Apache Zeppelin:** For interactive notebooks to visualize Spark queries and results.
- **Tableau or Power BI** (optional): For advanced data visualization and reporting (depending on project requirements).

Model Deployment (Implicit - Desktop/Web Application):

- No explicit deployment framework; however, the project can be developed to run on a local Hadoop/Spark cluster for analysis. Python installation is required on the target machine for executing the analysis and visualizing results.

3.3.3 Hardware Requirements

Processor:

- Minimum: **Intel Core i5** or equivalent
- Recommended: **Intel Core i7** or higher, for faster data processing with Spark.

Memory (RAM):

- Minimum: **8 GB RAM**
- Recommended: **16 GB RAM** or more, especially for handling large datasets efficiently in Spark.

Storage:

- Minimum: **500 GB HDD** (Preferably SSD for better performance)
- Recommended: **1 TB SSD** for faster data access and processing speed.

Graphics Processing Unit (GPU):

- Optional, not mandatory for most tasks, unless using Spark with GPU-acceleration for deep learning models.

Network:

- Stable internet connection for Hadoop cluster setup (if distributed processing is used).
- High-speed local network for accessing and transferring data between nodes in the cluster.

Other:

- **Monitor**: A screen with at least **1080p resolution** for clear data visualization and development.
- **Backup Storage**: External storage (e.g., 1 TB or more) for backup and data redundancy.

Implementation

Using Hive

1. Creation Of Table
CREATE TABLE banking_transactions (customer_id STRING, timestamp STRING, transaction_type STRING, amount FLOAT, location STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;

2. Loading data into the table
LOAD DATA INPATH 'data.csv' INTO TABLE banking_transactions;

3. Describe the table
desc banking_transactions;

4. Count the number of rows
select count(*) from branking_transactions;

5. Find the highest spending customers
SELECT customer_id, SUM(amount) AS total_spent FROM banking_transactions GROUP BY customer id ORDER BY total spent DESC LIMIT 3;

Using PySpark:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# Initialize Spark session
spark = SparkSession.builder.appName("FilterTransactions").getOrCreate()

# Load data from CSV
df = spark.read.csv("data.csv", header=True, inferSchema=True)

# Print before filtering
print("Data before filtering:")
df.show()

# Filter: Keep transactions greater than 500
df_filtered_1 = df.filter(col("amount") > 500)

# Print after first filter
print("Data after filtering for transactions greater than 500:")
df_filtered_1.show()

# Filter: Keep customers from a specific location (e.g., 'New York')
df_filtered_2 = df_filtered_1.filter(col("location") == "New York")
```

```python
# Print after second filter
print("Data after filtering for transactions greater than 500 and location as
New York:")
df_filtered_2.show()

# Additional filter: Keep transactions where the amount is not null
df_filtered_3 = df_filtered_2.filter(col("amount").isNotNull())

# Print final filtered data
print("Data after filtering for valid (non-null) transaction amounts:")
df_filtered_3.show()
```

Results and Discussion

5.1 Dataset

1. Transaction ID

Each transaction in the dataset is uniquely identified by a 'transaction id', which is typically an alphanumeric code. This field is crucial for ensuring that every transaction is traceable and non-duplicative. It serves as the primary key in data processing and is used for filtering or joining operations when integrating multiple data sources (e.g., customer info or fraud logs).

2. Account Number

The 'account number' is a unique identifier linked to a customer or user of the banking service. It allows the system to track all transactions associated with a specific account over time. In analytics, this field is critical for analyzing individual account behavior, such as frequency of transactions, total volume, or irregular activity. Though anonymized for privacy in many datasets, this field enables customer-level trend aggregation.

3. Transaction Type

This field classifies whether the transaction is a withdrawal or a deposit. It is usually categorical and can sometimes include other types like transfer, payment, or inquiry. For your project, this is a key variable since the goal is to analyze trends in withdrawals and deposits over time. Analysis might include identifying which transaction type is more frequent, time-based clustering, or demographic variations.

4. Transaction Amount

The 'Transaction amount' reflects the monetary value involved in each transaction. This numeric feature is essential for understanding financial activity—e.g., average withdrawal amount, high-value deposits, or spending behavior. It is also a critical factor in fraud detection, where unusually large transactions or out-of-pattern amounts can indicate anomalies.

5. Timestamp

The 'Timestamp' records the date and time the transaction was executed. This feature enables time-series analysis, helping to detect hourly, daily, weekly, or monthly patterns. It's used to track transaction trends such as peak activity periods, holiday behavior, end-of-month spikes, or even anomalies like duplicate transactions occurring within milliseconds of each other.

6. Location or Branch

The 'Location' or 'Branch' field provides geographic context to the transaction, such as the city, state, or specific bank branch where the transaction occurred. This is particularly helpful in regional trend analysis, helping banks understand branch-level performance or detect cross-location fraud (e.g., multiple transactions in distant locations within short time intervals).

7. Transaction Method

This optional field specifies how the transaction was carried out—whether through an ATM, mobile banking, web portal, POS machine, or direct teller service. It adds another dimension to behavioral analysis, indicating customer preferences or technology adoption trends. For example, a surge in mobile transactions may reflect a growing preference for digital banking solutions.

5.2 Explanation of Results

Hive Commands

1. **Table Creation:**
   The CREATE TABLE command creates a table in Hive called banking_transactions with the following fields:
   - customer_id: Stores customer identifiers (strings).
   - timestamp: The date and time of the transaction (string format).
   - transaction_type: The type of transaction (string).
   - amount: The amount of the transaction (floating-point number).
   - location: The location of the transaction (string).

The data is stored in text files, with fields delimited by commas.

2. **Loading Data:**
   The LOAD DATA INPATH 'data.csv' INTO TABLE banking_transactions; command loads data from the data.csv file into the banking_transactions table. The CSV file should be located in the specified directory. The columns in the CSV file are expected to match the order and types of the columns in the Hive table.
3. **Describe the Table:**
   The desc banking_transactions; command provides the schema of the banking_transactions table, showing details like column names, data types, and additional information such as whether null values are allowed.
4. **Counting Rows:**
   The query select count(*) from banking_transactions; counts the total number of rows in the banking_transactions table. This gives you an idea of how many transactions are present in the table.
5. **Highest Spending Customers:**
   The query:

```
sql
CopyEdit
SELECT customer_id, SUM(amount) AS total_spent
FROM banking_transactions
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 3;
```

This command selects the top 3 customers based on the total amount spent. It groups the data by customer_id, calculates the sum of the amount for each customer, and orders the result by total_spent in descending order. The LIMIT 3 ensures that only the top 3 customers with the highest total spend are returned.

PySpark Code

1. **Initialize Spark Session:**
   The first line initializes a PySpark session called spark with the application name "FilterTransactions".
2. **Loading Data from CSV:**
   The df = spark.read.csv("data.csv", header=True, inferSchema=True) command loads the data.csv file into a DataFrame (df), assuming the first row contains column headers. The inferSchema=True option allows Spark to automatically infer the data types of each column.
3. **Initial Data Preview:**

The df.show() command prints the first few rows of the dataset. This allows you to inspect the data before any filtering is done.

4. **First Filter - Transactions Greater Than 500:**
   The line df_filtered_1 = df.filter(col("amount") > 500) filters the transactions where the amount is greater than 500. This results in a DataFrame containing only transactions with amounts exceeding 500.
5. **Second Filter - Location:**
   The line df_filtered_2 = df_filtered_1.filter(col("location") == "New York") further filters the DataFrame to include only transactions from the location "New York". This results in a DataFrame containing only high-value transactions (greater than 500) that occurred in New York.
6. **Third Filter - Non-Null Amounts:**
   The line df_filtered_3 = df_filtered_2.filter(col("amount").isNotNull()) ensures that the transaction amounts are not null. It filters out any rows where the amount is null, leaving only rows with valid amounts.
7. **Final Data Preview:**
   The final df_filtered_3.show() prints the remaining data after all the filters have been applied.This is the final DataFrame containing transactions greater than 500, from NewYork

5.3 Results

```
hive> create table banking_transactions(customer_id string,timestamp string,tra
saction_type string,amount float,location string)row format delimited fields te
minated by ',' stored as textfile;
OK
Time taken: 0.114 seconds
```

Figure 1 : Table creation using Hive commands

```
hive> load data inpath 'data.csv' into table banking_transactions;
Loading data to table default.banking_transactions
Table default.banking_transactions stats: [numFiles=1, totalSize=39708]
OK
Time taken: 0.901 seconds
```

Figure 2 : Loading data

```
hive> desc banking_transactions;
OK
customer_id             string
timestamp               string
transaction_type        string
amount                  float
location                string
Time taken: 0.14 seconds, Fetched: 5 row(s)
```

Figure 3 : Description of table

```
MapReduce Total cumulative CPU time: 3 seconds 940 msec
Ended Job = job_1745927674712_0001
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 3.94 sec   HDFS Read: 47661 HDFS Write: 4
Total MapReduce CPU Time Spent: 3 seconds 940 msec
OK
765
Time taken: 45.919 seconds, Fetched: 1 row(s)
```

Figure 4 : Counting rows using MapReduce

```
MapReduce Total cumulative CPU time: 2 seconds 250 msec
Ended Job = job_1745927674712_0003
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 3.42 sec   HDFS Read: 47418 HDFS Write: 3203 SUCC
ESS
Stage-Stage-2: Map: 1  Reduce: 1   Cumulative CPU: 2.25 sec   HDFS Read: 8286 HDFS Write: 75 SUCCESS
Total MapReduce CPU Time Spent: 5 seconds 670 msec
OK
10038   39293.309997558594
10015   37787.389709472656
10083   37689.260314941406
Time taken: 65.072 seconds. Fetched: 3 row(s)
```

Figure 5 : List of the highest spending customers

```
Data before filtering:
+-----------+-------------------+----------------+-------+-------------+
|customer_id|          timestamp|transaction_type| amount|     location|
+-----------+-------------------+----------------+-------+-------------+
|      10054|2024-05-15 10:21:39|         deposit|2304.29|      Chicago|
|      10007|2025-01-15 02:32:26|      withdrawal| 679.52|       Boston|
|      10081|2024-02-26 18:57:17|         deposit|4290.98|      Houston|
|      10015|2024-05-12 07:13:41|      withdrawal| 500.86|        Miami|
|      10059|2025-01-27 20:30:25|         deposit|4602.56|San Francisco|
|      10048|2024-04-11 06:50:16|         deposit|1998.29|  Los Angeles|
|      10009|2025-01-14 09:53:27|         deposit|2959.22|       Dallas|
|      10056|2025-01-03 14:13:42|      withdrawal|4136.01|  Los Angeles|
|      10012|2024-03-20 22:50:12|      withdrawal|4077.05|       Dallas|
|      10024|2024-08-29 02:40:42|         deposit|3997.81|       Boston|
|      10030|2024-07-15 21:27:51|      withdrawal| 853.75|       Dallas|
|      10079|2024-03-07 19:51:03|         deposit|  363.7|      Chicago|
|      10054|2024-11-23 02:57:19|         deposit| 1420.3|  Los Angeles|
|      10096|2024-08-22 10:21:28|      withdrawal|1440.19|      Chicago|
|      10021|2024-05-26 01:54:00|         deposit|3513.15|  Los Angeles|
|      10086|2024-08-08 19:48:46|      withdrawal|2474.86|      Chicago|
|      10093|2024-05-14 02:32:13|      withdrawal|1204.14|       Boston|
|      10065|2024-06-20 21:31:31|         deposit|1380.95|  Los Angeles|
|      10063|2024-02-15 15:55:50|      withdrawal|3042.81|      Atlanta|
|      10011|2024-12-21 12:34:25|      withdrawal| 165.22|        Miami|
+-----------+-------------------+----------------+-------+-------------+
only showing top 20 rows
```

Figure 6 : Unfiltered data

```
Data after filtering for transactions greater than 500:
+-----------+-------------------+----------------+-------+-------------+
|customer_id|          timestamp|transaction_type| amount|     location|
+-----------+-------------------+----------------+-------+-------------+
|      10054|2024-05-15 10:21:39|         deposit|2304.29|      Chicago|
|      10007|2025-01-15 02:32:26|      withdrawal| 679.52|       Boston|
|      10081|2024-02-26 18:57:17|         deposit|4290.98|      Houston|
|      10015|2024-05-12 07:13:41|      withdrawal| 500.86|        Miami|
|      10059|2025-01-27 20:30:25|         deposit|4602.56|San Francisco|
|      10048|2024-04-11 06:50:16|         deposit|1998.29|  Los Angeles|
|      10009|2025-01-14 09:53:27|         deposit|2959.22|       Dallas|
|      10056|2025-01-03 14:13:42|      withdrawal|4136.01|  Los Angeles|
|      10012|2024-03-20 22:50:12|      withdrawal|4077.05|       Dallas|
|      10024|2024-08-29 02:40:42|         deposit|3997.81|       Boston|
|      10030|2024-07-15 21:27:51|      withdrawal| 853.75|       Dallas|
|      10054|2024-11-23 02:57:19|         deposit| 1420.3|  Los Angeles|
|      10096|2024-08-22 10:21:28|      withdrawal|1440.19|      Chicago|
|      10021|2024-05-26 01:54:00|         deposit|3513.15|  Los Angeles|
|      10086|2024-08-08 19:48:46|      withdrawal|2474.86|      Chicago|
|      10093|2024-05-14 02:32:13|      withdrawal|1204.14|       Boston|
|      10065|2024-06-20 21:31:31|         deposit|1380.95|  Los Angeles|
|      10063|2024-02-15 15:55:50|      withdrawal|3042.81|      Atlanta|
|      10061|2025-03-19 05:34:33|         deposit|4230.11|      Houston|
|      10077|2025-01-07 13:02:47|         deposit|3862.53|      Seattle|
+-----------+-------------------+----------------+-------+-------------+
only showing top 20 rows
```

Figure 7 : First Filter - Transactions Greater Than 500

```
Data after filtering for transactions greater than 500 and location as New York:
+-----------+-------------------+----------------+-------+--------+
|customer_id|          timestamp|transaction_type| amount|location|
+-----------+-------------------+----------------+-------+--------+
|      10071|2025-02-25 05:21:29|      withdrawal|4294.94|New York|
|      10055|2024-07-25 16:22:40|      withdrawal|4846.74|New York|
|      10071|2024-02-29 08:22:16|      withdrawal|2137.25|New York|
|      10039|2024-06-20 20:01:41|         deposit|2049.28|New York|
|      10049|2025-02-21 21:00:37|         deposit|1873.82|New York|
|      10019|2024-01-15 11:56:17|      withdrawal|3863.94|New York|
|      10046|2024-05-26 11:55:51|         deposit|3135.22|New York|
|      10027|2025-01-19 12:45:11|         deposit|1771.65|New York|
|      10016|2024-04-24 20:00:00|         deposit|4224.96|New York|
|      10014|2024-05-22 11:17:07|      withdrawal|4727.54|New York|
|      10096|2024-03-26 20:47:30|      withdrawal|1445.34|New York|
|      10063|2024-12-03 12:55:04|      withdrawal| 665.37|New York|
|      10011|2024-06-09 11:09:23|         deposit|4381.32|New York|
|      10021|2025-01-04 14:15:39|      withdrawal|3620.93|New York|
|      10066|2024-04-15 13:54:50|      withdrawal|4957.48|New York|
|      10070|2024-01-15 20:54:06|         deposit|3489.24|New York|
|      10003|2024-03-29 12:54:18|      withdrawal| 3807.0|New York|
|      10003|2024-03-27 17:20:05|      withdrawal|2492.51|New York|
|      10096|2024-08-09 00:28:34|      withdrawal|4122.49|New York|
|      10035|2024-04-10 01:32:14|         deposit| 791.49|New York|
+-----------+-------------------+----------------+-------+--------+
only showing top 20 rows
```

Figure 7 : Second Filter – Location

```
Data after filtering for valid (non-null) transaction amounts:
+-----------+-------------------+----------------+-------+--------+
|customer_id|          timestamp|transaction_type| amount|location|
+-----------+-------------------+----------------+-------+--------+
|      10071|2025-02-25 05:21:29|      withdrawal|4294.94|New York|
|      10055|2024-07-25 16:22:40|      withdrawal|4846.74|New York|
|      10071|2024-02-29 08:22:16|      withdrawal|2137.25|New York|
|      10039|2024-06-20 20:01:41|         deposit|2049.28|New York|
|      10049|2025-02-21 21:00:37|         deposit|1873.82|New York|
|      10019|2024-01-15 11:56:17|      withdrawal|3863.94|New York|
|      10046|2024-05-26 11:55:51|         deposit|3135.22|New York|
|      10027|2025-01-19 12:45:11|         deposit|1771.65|New York|
|      10016|2024-04-24 20:00:00|         deposit|4224.96|New York|
|      10014|2024-05-22 11:17:07|      withdrawal|4727.54|New York|
|      10096|2024-03-26 20:47:30|      withdrawal|1445.34|New York|
|      10063|2024-12-03 12:55:04|      withdrawal| 665.37|New York|
|      10011|2024-06-09 11:09:23|         deposit|4381.32|New York|
|      10021|2025-01-04 14:15:39|      withdrawal|3620.93|New York|
|      10066|2024-04-15 13:54:50|      withdrawal|4957.48|New York|
|      10070|2024-01-15 20:54:06|         deposit|3489.24|New York|
|      10003|2024-03-29 12:54:18|      withdrawal| 3807.0|New York|
|      10003|2024-03-27 17:20:05|      withdrawal|2492.51|New York|
|      10096|2024-08-09 00:28:34|      withdrawal|4122.49|New York|
|      10035|2024-04-10 01:32:14|         deposit| 791.49|New York|
+-----------+-------------------+----------------+-------+--------+
only showing top 20 rows
```

Figure 8 : Third Filter - Non-Null Amounts


```
Data after filtering for valid (non-null) transaction amounts:
+-----------+-------------------+----------------+-------+--------+
|customer_id|          timestamp|transaction_type| amount|location|
+-----------+-------------------+----------------+-------+--------+
|      10071|2025-02-25 05:21:29|      withdrawal|4294.94|New York|
|      10055|2024-07-25 16:22:40|      withdrawal|4846.74|New York|
|      10071|2024-02-29 08:22:16|      withdrawal|2137.25|New York|
|      10039|2024-06-20 20:01:41|         deposit|2049.28|New York|
|      10049|2025-02-21 21:00:37|         deposit|1873.82|New York|
|      10019|2024-01-15 11:56:17|      withdrawal|3863.94|New York|
|      10046|2024-05-26 11:55:51|         deposit|3135.22|New York|
|      10027|2025-01-19 12:45:11|         deposit|1771.65|New York|
|      10016|2024-04-24 20:00:00|         deposit|4224.96|New York|
|      10014|2024-05-22 11:17:07|      withdrawal|4727.54|New York|
|      10096|2024-03-26 20:47:30|      withdrawal|1445.34|New York|
|      10063|2024-12-03 12:55:04|      withdrawal| 665.37|New York|
|      10011|2024-06-09 11:09:23|         deposit|4381.32|New York|
|      10021|2025-01-04 14:15:39|      withdrawal|3620.93|New York|
|      10066|2024-04-15 13:54:50|      withdrawal|4957.48|New York|
|      10070|2024-01-15 20:54:06|         deposit|3489.24|New York|
|      10003|2024-03-29 12:54:18|      withdrawal| 3807.0|New York|
|      10003|2024-03-27 17:20:05|      withdrawal|2492.51|New York|
|      10096|2024-08-09 00:28:34|      withdrawal|4122.49|New York|
|      10035|2024-04-10 01:32:14|         deposit| 791.49|New York|
+-----------+-------------------+----------------+-------+--------+
only showing top 20 rows
```


Figure 9 : Third Filter - Non-Null Amounts

```
C:\bda>SUCCESS: The process with PID 3592 (child process of PID 27856) has been terminated.
SUCCESS: The process with PID 27856 (child process of PID 19348) has been terminated.
SUCCESS: The process with PID 19348 (child process of PID 15888) has been terminated.
```

Figure 10 : Final data preview

Conclusion

This project demonstrated the effective integration of Hive and PySpark for analyzing and filtering banking transaction data, leveraging the strengths of both technologies to process large-scale datasets efficiently. The main goal was to utilize these tools for data storage, transformation, and aggregation, ultimately extracting meaningful insights from banking transactions. By combining Hive for structured data storage and PySpark for real-time data processing, we were able to perform operations that are both time-efficient and scalable.

Hive was used primarily for data storage and SQL-based querying, enabling us to create a table to hold banking transaction data and perform basic operations like counting rows and identifying top-spending customers. Hive's table structure is well-suited for managing large datasets, and its SQL-like syntax allowed for simple yet powerful data management tasks. We loaded data from a CSV file into Hive, which allowed for easy handling of large datasets with minimal setup. Furthermore, using Hive's SQL syntax enabled us to run aggregation queries, such as counting the total number of rows or selecting customers with the highest total spending.

On the other hand, PySpark was employed for more advanced data processing tasks, leveraging its distributed computing capabilities. PySpark allowed us to load the transaction data from a CSV file into a DataFrame and perform a series of filtering operations. These included selecting transactions where the amount was greater than 500, restricting the data to a specific location (e.g., New York), and ensuring that only non-null transaction amounts were considered in the analysis. PySpark's flexibility with its DataFrame API allowed us to apply multiple filters and transformations, enabling granular data analysis. Additionally, PySpark can efficiently process data in parallel, making it suitable for working with large datasets in a distributed environment.

One key advantage of using PySpark was its ability to handle real-time data transformations. With just a few lines of code, we could apply multiple filters on the data, transforming it into a format that best suited our analysis needs. The ability to chain multiple filters and transformations made PySpark an ideal choice for processing large-scale transactional data.

Together, Hive and PySpark form a powerful combination for managing and analyzing banking transaction data. This project demonstrated how businesses can leverage these technologies to extract valuable insights such as identifying high-value customers, analyzing spending patterns, and tracking location-specific transactions. It also showcased the scalability of both tools, with Hive efficiently storing and managing large datasets, while PySpark offered fast, distributed processing of data, allowing for real-time insights. Overall, this project serves as a practical example of how Hive and PySpark can be used together to tackle data processing challenges in a real-world scenario.

References

[1] Apache Hive Documentation, Apache Software Foundation
Link:https://cwiki.apache.org/confluence/display/Hive/Home

[2] PySpark Documentation, Apache Spark
Link: https://spark.apache.org/docs/latest/api/python/

[3] Hive SQL Language Reference, Apache Software Foundation
Link: https://cwiki.apache.org/confluence/display/Hive/LanguageManual

[4] Learning PySpark: Building Python-Based Distributed Data Analytics Solutions with Spark, Tomasz Drabas, Denny Lee
Link: https://www.packtpub.com/product/learning-pyspark/9781788621757

[5] Big Data Analytics with Spark: A Hands-On Approach, Venkat Ankam
Link: https://www.apress.com/gp/book/9781484212329

[6] Apache Hive – The Definitive Guide, Dean Wampler, Jason Rutherglen, and Edward Capriolo
Link: https://www.oreilly.com/library/view/apache-hive-the/9781449319250/

[7] Mastering Apache Spark 2.x, Romeo Kienzler
Link: https://www.packtpub.com/product/mastering-apache-spark-2x/9781788621757

[8] Learning Spark: Lightning-Fast Big Data Analysis, Holden Karau, Andy Konwinski, Patrick Wendell, Matei Zaharia
Link: https://www.oreilly.com/library/view/learning-spark/9781449358624/

[9] Practical Hive: A Guide to Hadoop's Data Warehouse System, Ankur Goel
Link: https://www.apress.com/gp/book/9781430247690

[10] Spark: The Definitive Guide: Big Data Processing Made Simple, Bill Chambers, Matei Zaharia
Link: https://www.oreilly.com/library/view/spark-the-definitive/9781491912218/